

Course Outline Summer 2024

Object Oriented Programming

SCU	4 Credit(s)
Co-requisite (s)	
Pre-requisite(s)	a) Programming Fundamental (CS-1012) b) Programming Fundamental–II (CS-1013) c) Computer Programming (CS-1101)
Weekly tuition pattern	2 Class Sessions (90 min each) 1 Lab Session (180 min)
Teaching Team	Dr. Muhammad Atif, Ms. Iqra Adnan, Mr. Sardar Waqar, Ms. Mariya Bibi, Mr. Ali Raza, Dr. Ashfaq, Mr. Muhammad Rehan, Ms. Kashmala Pervaiz.

1-Course Description

Programmers face problem of modeling real world artifacts in the programming solution when they move from writing small programs to bigger software applications. Conventional programming methods fail to resolve this issue. This is where you need to be aware of object-oriented programming to overcome the solution complexities. It takes lots of hours to write well organized, less complex, scalable and reusable quality programs. What OOP (object-oriented programming) stands for, is just a way to properly organize your code into logical components that are easier to program, manage and enhance.

This course is designed to familiarize students with basic concepts of OOP like Classes, Objects and other main characteristics to write OO programs.

2-Objectives

The goal of this course is to:

- 1) Provide sound understanding of OOP concepts.
- 2) Provide students a comparison of structured and Object-oriented programming

- techniques.
- 3) Teach students how to transform real world scenarios in OO model.
 - 4) Provide students with the knowledge of relations between classes such as inheritance and other OOP features.
 - 5) Store and retrieve data from / on disk.

3- Course Learning Outcome

- 1) Understand the relative merits of C++ as an object-oriented programming language
- 2) Understand the features of C++ supporting object-oriented program
- 3) Understand how to produce object-oriented software using C++
- 4) Understand how to apply the major object-oriented concepts to implement object-oriented programs in C++, encapsulation, inheritance and polymorphism
- 5) Understand advanced features of C++ specifically stream I/O, templates and operator overloading

4- Course Structure

1.	Lectures
2.	Class Activities and lab assignments
3.	Group project
4.	Quiz

5- Course Duration

This course will be held twice a week of 3-hour class duration.

6- Course style

The course will be delivered in a classroom environment.

7- Additional Course Requirement

In addition to the objectives of this course, students are expected to gain skills which would be needed in the professional work environment. These skills include but not limited to: Writing, Presentation, Decision Making and Teamwork.

8- Text and Other Resources

8.1 Text

<i>Robert Lafore, Object-Oriented Programming in C++</i>
--

8.2 Other Resources

<i>Harvey & Paul Deitel & Deitel, C++ How to Program, 5th Edition, Prentice Hall</i>
<i>Bjarne Stroustrup, The C++ Programming Language, 3rd Edition</i>

9- Course Outline

The lecturers are advised to complete the following topics/sub-topics before the mid/final term examination as prescribed in the course outline below:

Week	Lecture #	Topics/Sub-Topics
1	Lecture 1	Course Objectives. Introduction of Object-oriented programming, Structured vs Object Oriented Programming.
	Lecture 2	Revision of programming concepts (Types of Functions, Built-in, User defined, passing parameters to Functions by value and by Reference,)
2	Lecture 3	Default Parameters, Function Overloading, Inline Functions. OOP and Real-World Modeling. Need of OOP (Primitive and User define Data type, ADT). Basic features of OOP (Introduction to abstraction, Encapsulation, Information Hiding)
	Lecture 4	Class, Difference between Class and Structures Declaring a Class. Creating an Object, Properties of an object, Relationship between objects and classes.
3	Lecture 5	Type of Access Specifiers in class, Usage of Access Specifiers, Implementation of Data Encapsulation, & Information Hiding in Classes, Abstraction
	Lecture 6	Defining Member functions Inside and Outside a class, Accessing member Functions, Constant member functions
4	Lecture 7	Static data member, static function member, Objects as function Parameter
	Lecture 8	Constructors, Need of constructor, default constructor, Parameterized constructor
5	Lecture 9	Copy constructor, Constant reference parameters,
	Lecture 10	Setters and Getters
6	Lecture 11	Pointers, Dynamic memory allocation, Destructor, Similarities and differences between constructor and Destructor
	Lecture 12	Introduction to Inheritance. Purpose of Inheritance, Inheritance Advantages, Aggregation & Composition (Fundamentals, Differences & UML representation)
7	Lecture 13	Introduction to UML notations of class
	Lecture 14	Inheritance Categories (Single, Multilevel, Multiple), Implementing Single Inheritance.
8	Lecture 15	Use of public Access Specifier for inheritance (Accessing members of Base (Parent) and Derived (Child) Class) Use of Private Access Specifier for inheritance (Accessing members of Base (Parent) and Derived (Child) Class) Use of Protected Access Specifier for inheritance (Accessing members of Base (Parent) and Derived (Child) Class) Next implementing person student class
	Lecture 16	Use of Default Constructor, in single inheritance only Parameterized constructors and Destructors in context of inheritance, Difference between Function Overloading and Function Overriding
9	Mid Term Exam	

10	Lecture 17	Multilevel Inheritance, Implementing Multilevel inheritance, Sequence of constructor and Destructor
	Lecture 18	Multiple Inheritance, Need of Multiple Inheritances, Implementing Multiple Inheritance. Diamond Problem
11	Lecture 19	Friend functions, friend classes Virtual Inheritance
	Lecture 20	Declaring Pointers to an object, Inheritance hierarchy and pointers Base, class pointer to Derived class, Dynamic memory, dangling object
12	Lecture 21	Introduction to Polymorphism. Implementing Polymorphism, Introduction to Virtual functions
	Lecture 22	Introducing & Implementing Compile Time (Static) Polymorphism, Repeat Function Overloading
13	Lecture 23	Introducing & Implementing Run Time Polymorphism (Dynamic Polymorphism), Virtual Base Classes, Abstract class, Abstract class and Pure virtual function
	Lecture 24	Operator Overloading: using member functions (Unary and binary)
14	Lecture 25	Operator Overloading: using non-member functions (friend functions) unary and binary
	Lecture 26	Shallow copy and deep copy: Copy constructor & Assignment operator overloading
15	Lecture 27	File streaming: Text files
	Lecture 28	File streaming: binary files
16	Lecture 29	Introduction to Template functions
	Lecture 30	Introduction to Exception Handling, Need of Exception Handling
Final Term Exam		

10- Assessment Criteria

No.	Assessment	Percentage
1.	Mid Exam	20%
2.	Final Exam	40%
3.	Quiz	10%
4.	Programming Assignments	10%
5.	Lab	20%
Total		100%

11- Attendance Requirements

You are expected to attend all lectures, seminars, tutorials, and lab sessions or any other classroom activity. Where you fail to attend classes, you cannot expect the lecturer to brief you on what you have missed. You are responsible for your attendance, not the academic staff. Attendance at tutorials and lab sessions will be strictly monitored, and failure to attend will be taken into account.

12- Submission and Collection of Assignment

All assignments should be handed in at the beginning of the class sessions when they are due. All assignments may be handed back during scheduled classes.

13- General Information

Students are required to be familiar with the university code of conduct, and to abide by its terms and conditions.

13.1 Copying of Copyright Material by a Student

A condition of acceptance as a student is the obligation to abide by the University's policy on the copying of copyright material. This obligation covers photocopying of any material using the University's photocopying machines, and the recording off air, and making subsequent copies, of radio or television broadcasts, and photocopying textbooks. Students who flagrantly disregard University policy and copyright requirements will be liable to disciplinary action under the Code of Conduct.

13.2 Academic Misconduct

Please refer to the Code of Conduct for definitions and penalties for Academic Misconduct, plagiarism, collusion, and other specific acts of academic dishonesty. A student may work together with one or a group of students discussing assignment content, identifying relevant references, and debating issues relevant to the subject. Encouraging or assisting another person to commit plagiarism is a form of improper collusion and may attract the same penalties. To prevent Academic Misconduct occurring, students are expected to familiarize themselves with the University policy, the Subject Outline statements, and specific assignment guidelines. Students should also seek advice from Subject Leaders on acceptable academic conduct.

13.3 Guidelines to Avoid Plagiarism

Whenever you copy more than a few words from any source, you must acknowledge that source by putting the quote in quotation marks and providing the name of the author. Students are encouraged to co-operate, but collusion is a form of cheating. Students may work together in obtaining references, discussing the content of the references and discussing the assignment, but when they write, they must write alone.

Approval

Checked by,	Approved by, <u>Dr. Muhammad Atif</u> Cluster Head(OOP)
-------------	--

Last Update Date: 25th September, 2020